Readme_1: Análisis del Modelo y Generación de Datos Masivos

Proyecto Integrador _Bases de Datos

FleetLogix: Sistema de Gestión de Transporte y Logística

1. Descripción del Avance

El Avance 1 corresponde al análisis del modelo relacional y la generación de datos sintéticos masivos para el sistema FleetLogix, una plataforma de gestión de transporte y logística que opera con una flota de 200 vehículos en 5 ciudades mexicanas.

Este avance cubre dos componentes principales: el diseño y documentación del modelo relacional (diagrama ER), y la generación automatizada de más de 505,000 registros coherentes y realistas usando Python con las librerías Faker y pandas.

---

2. Objetivo

Poblar una base de datos PostgreSQL vacía con datos sintéticos que simulen 2 años de operación real de la empresa FleetLogix, respetando todas las reglas de integridad referencial y distribuciones de negocio realistas.

---

3. Modelo Relacional

3.1 Descripción de las 6 tablas

El esquema de base de datos está compuesto por 6 tablas que representan las entidades principales del negocio:

Tabla	Registros	Descripción
vehicles	200	Flota de vehículos: camiones, vans y motocicletas con capacidad, combustible y estado
drivers	400	Conductores con código de empleado, licencia, teléfono y fecha de contratación
routes	48	Rutas predefinidas entre las 5 ciudades mexicanas con distancia y duración estimada
trips	110,000	Viajes realizados: relaciona vehículo, conductor y ruta con tiempos y combustible
deliveries	400,000	Entregas individuales por viaje: tracking, cliente, dirección, peso y estado
maintenance	4,908	Historial de mantenimiento de vehículos: tipo, costo y próxima fecha

3.2 Relaciones (Foreign Keys)

La jerarquía de dependencias entre tablas define el orden de inserción y garantiza la integridad referencial:

- vehicles → trips (vehicle_id): cada viaje pertenece a un vehículo activo
- drivers → trips (driver_id): cada viaje es conducido por un conductor activo
- routes → trips (route_id): cada viaje sigue una ruta predefinida
- trips → deliveries (trip_id): cada entrega pertenece a un viaje
- vehicles → maintenance (vehicle_id): cada mantenimiento aplica a un vehículo

3.3 Decisiones de diseño

- Las tablas vehicles y drivers tienen columna status (active/maintenance o active/inactive) para filtrar registros válidos en queries operacionales
- trips.arrival_datetime puede ser NULL: representa viajes en progreso o cancelados
- deliveries.recipient_signature es BOOLEAN para facilitar KPIs de calidad de entrega
- maintenance.next_maintenance_date permite programar alertas preventivas

4. Script de Generación de Datos

4.1 Archivo

Archivo principal: **Avance_1_DataGeneration.ipynb** (Jupyter Notebook compatible con VS Code)

4.2 Tecnologías utilizadas

Librería	Versión	Uso en el proyecto
Faker (es_MX)	≥19.0	Nombres, direcciones y teléfonos en formato mexicano
pandas	≥2.0	DataFrames para manipulación y exportación de datos
numpy	≥1.24	Distribuciones estadísticas (exponencial, choice ponderado)
psycopg2-binary	≥2.9	Conexión a PostgreSQL e inserción con execute_batch
tqdm	≥4.65	Barras de progreso para monitorear generación masiva

4.3 Estructura del notebook

El notebook está organizado en 6 celdas de ejecución secuencial:

- Celda 1 — Instalación de dependencias (pip install)
- Celda 2 — Configuración: DB_CONFIG, SEED=42, Faker(es_MX)
- Celda 3 — Clase DataGenerator con 6 métodos de generación
- Celda 4 — Ejecución principal: genera las 6 tablas en orden
- Celda 5 — Exportación CSV y reporte JSON de validación
- Celda 6 — Exploración estadística de los datos generados

4.4 Reglas de negocio aplicadas

- Distribución de flota: 30% Camión Grande, 30% Camión Mediano, 30% Van, 10% Motocicleta
- Solo vehículos y conductores activos se asignan a viajes (status = active)
- El peso cargado en cada viaje es entre 40% y 90% de la capacidad del vehículo
- Distribución horaria realista: picos de 8-12h y 14-18h (mayor operación laboral)
- Número de entregas por viaje: distribución ponderada de 2-6 (promedio 4)
- Peajes en pesos mexicanos: $200 MXN por cada 100 km de distancia

- Mantenimiento: 1 registro por cada 20 viajes que realiza cada vehículo

---

5. Resultados de la Generación

5.1 Registros generados

Tabla	Registros	Tiempo aprox.	Observaciones
vehicles	200	< 1 seg	4 tipos, 90% activos
drivers	400	< 1 seg	95% activos, 2 apellidos
routes	48	< 1 seg	5 ciudades mexicanas
trips	110,000	~10 seg	Distribuidos en 2 años
deliveries	400,000	~55 seg	Promedio 4 por viaje
maintenance	4,908	< 5 seg	1 por cada 20 viajes
TOTAL	505,556	~70 seg	Objetivo: ≥ 505,000 ✓

5.2 Validaciones de integridad

El notebook ejecuta 5 validaciones automáticas al terminar la generación. Todos los resultados fueron 0 errores:

- FK: Trips sin vehicle_id valido — 0 errores
- FK: Deliveries sin trip_id valido — 0 errores
- Temporal: arrival_datetime < departure_datetime — 0 errores
- Peso: trips.total_weight_kg > vehicles.capacity_kg — 0 errores
- Deliveries sin tracking_number — 0 errores

6. Instrucciones de Ejecución

6.1 Prerrequisitos

- Python 3.10 o superior
- PostgreSQL 15 corriendo localmente en puerto 5432
- Base de datos fleetlogix creada con el DDL de fleetlogix_db_schema.sql
- VS Code con extensión Jupyter instalada

6.2 Pasos

- 1. Abrir Avance_1_DataGeneration.ipynb en VS Code
- 2. Se cambió el_password por la contraseña real de PostgreSQL
- 3. Verificar LOAD_TO_DB = True para insertar en la BD
- 4. Ejecutar celdas en orden con Shift+Enter
- 5. Al finalizar, revisar el archivo generation_summary.json para el reporte de validación

6.3 Salidas generadas

- data/vehicles.csv, drivers.csv, routes.csv, trips.csv, deliveries.csv, maintenance.csv
- generation_summary.json — reporte de conteos y validaciones
- data_generation.log — log detallado del proceso

---

7. Estructura del Proyecto

```
FleetLogix/
  Avance_1_DataGeneration.ipynb   <- Notebook principal
  fleetlogix_db_schema.sql         <- DDL: CREATE TABLE e indices
  generation_summary.json          <- Reporte de validacion
  data_generation.log              <- Log de ejecucion
  data/
    vehicles.csv                   <- 200 registros
    drivers.csv                    <- 400 registros
    routes.csv                     <- 48 registros
    trips.csv                      <- 110,000 registros
    deliveries.csv                 <- 400,000 registros
    maintenance.csv                <- 4,908 registros
```

---

8. Decisiones Técnicas Relevantes

¿Por qué Faker con locale es_MX?

El proyecto opera en Mexico. El local es_MX genera nombres, apellidos y direcciones en formato mexicano, haciendo los datos coherentes con las ciudades de operación (CDMX, Guadalajara, Monterrey, Puebla, Tijuana).

¿Por qué execute_batch en lugar de INSERT fila por fila?

execute_batch de psycopg2 inserta registros en lotes (page_size=100), reduciendo el número de round-trips a la BD. Para 110,000 trips, esto es aproximadamente 1,000 veces más rápido que 110,000 INSERTs individuales.

¿Por qué SEED = 42?

Fijar la semilla en random, numpy y Faker garantiza que ejecutar el script dos veces produce exactamente los mismos datos. Esto es critico para que todo el equipo trabaje con la misma BD y para reproducibilidad en evaluaciones.

¿Por qué la clase DataGenerator en lugar de funciones sueltas?

La clase encapsula la conexión a PostgreSQL (self.connection, self.cursor) y los DataFrames generados (self.dataframes) en un solo objeto. Esto evita pasar parametros entre funciones y facilita el manejo de errores con try/finally.